COS341 2026: Continuation of **Semester-Practical-Specification** Part 2**b**) ***Type Analysis***

General Remark:
In this phase we assume that *all user-defined names have already been consistently replaced* by *system-generated new names* (in the foregoing phase 2**a**) such that:

- IF two syntactic elements have the same system-generated new name, then they are also semantically the same entity
- IF two syntactic elements are semantically different entities then they carry different system-generated new names
- All this information is already available in a suitable *symbol table.*

Now, in this phase (2b) of the project, we will fill the symbol table with ***type information*** according to the following ***semantic rules:***

Initially, the type-information *attributed* to *every node* of the syntax tree is "unknown". *During the work of your type analyser the* "unknown" *attributes will get replaced* by more specific information until the entire syntax tree has been completely visited.

For each syntactic rule of the SPL Grammar, some hints are given about its intended type semantics. It is *your task* to "understand" these hints and to *properly implement* your type-analyser accordingly. Appropriate *error-messages* must be "thrown" in case that a given *syntax-tree violates* any semantic rule.

SPL_PROG → P$ *if* type_of(P) is "ok", *then* **update**: let type_of(SPL_PROG) be "ok".

P → V_DECL **:** F_DECL **:** ALGO
if type_of(V_DECL) is "ok"
and *if* type_of(F_DECL) is "ok"
and *if* type_of(ALGO) is "ok",
then **update**: let type_of(P) be "ok".

V_DECL → ε // *nullable* **update**: let type_of(V_DECL) be "ok".

V_DECL → **NAME** V_DECL
if type_of(V_DECL) is "ok",
then **update**: let type_of(**NAME**) be "**numeric**" ;
let type_of(V_DECL) be "ok".

F_DECL → ε // *nullable* **update**: let type_of(F_DECL) be "ok".

F_DECL → F_TYPE F_DECL
if type_of(F_TYPE) is "ok"
and *if* type_of(F_DECL) is "ok",
then **update**: let type_of(F_DECL) be "ok".

F_TYPE → **void NAME (** V_DECL **) { ** P **return }**
if type_of(P) is "ok"
and *if* type_of(V_DECL) is "ok",
then **update**: let type_of(**NAME**) be "**procedure**" ;
let type_of(F_TYPE) be "ok".

---

turn the page ►

F_TYPE	→	**num** **NAME** **(** V_DECL **) { P return (** TERM **) }**	*if* type_of(P) is "ok" and *if* type_of(V_DECL) is "ok" and *if* type_of(TERM) is "numeric", *then* **update**: let type_of(**NAME**) be "**numeric**" ; let type_of(F_TYPE) be "ok".
ALGO	→	ε // *nullable*	**update**: let type_of(ALGO) be "ok".
ALGO	→	INSTR **;** ALGO	*if* type_of(ALGO) is "ok" and *if* type_of(INSTR) is "ok", *then* **update**: let type_of(ALGO) be "ok".
OUTP	→	**(** TERM **)**	*if* type_of(TERM) is "numeric", *then* **update**: let type_of(OUTP) be "ok".
OUTP	→	**STRING**	**update**: let type_of(OUTP) be "ok".
INSTR	→	**print** OUTP	*if* type_of(OUTP) is "ok", *then* **update**: let type_of(INSTR) be "ok".
INSTR	→	**nop**	**update**: let type_of(INSTR) be "ok".
INSTR	→	**comment** **STRING**	**update**: let type_of(INSTR) be "ok".
INSTR	→	ASSIGN	*if* type_of(ASSIGN) is "ok", *then* **update**: let type_of(INSTR) be "ok".
INSTR	→	BRANCH	*if* type_of(BRANCH) is "ok", *then* **update**: let type_of(INSTR) be "ok".
INSTR	→	LOOP	*if* type_of(LOOP) is "ok", *then* **update**: let type_of(INSTR) be "ok".
INSTR	→	CALL	*if* type_of(CALL) is "procedure", *then* **update**: let type_of(INSTR) be "ok".
CALL	→	**NAME** **(** INPUT **)**	*if* type_of(INPUT) is "ok" and *if* type_of(**NAME**) =/= "unknown", *then* **update**: let type_of(CALL) be = type_of(**NAME**).
INPUT	→	ε // *nullable*	**update**: let type_of(INPUT) be "ok".
INPUT	→	TERM INPUT	*if* type_of(TERM) is "numeric" and *if* type_of(INPUT) is "ok", *then* **update**: let type_of(INPUT) be "ok".
ASSIGN	→	**NAME** = TERM	*if* type_of(TERM) is "numeric" and *if* type_of(**NAME**) is "numeric", *then* **update**: let type_of(ASSIGN) be "ok".
TERM	→	**NAME**	*if* type_of(**NAME**) is "numeric", *then* **update**: let type_of(TERM) be "**numeric**".
TERM	→	**NUM**	**update**: let type_of(TERM) be "**numeric**".
TERM	→	CALL	*if* type_of(CALL) is "numeric", *then* **update**: let type_of(TERM) be "**numeric**".

turn the page ►

TERM → **mod (TERM TERM)**
TERM → **div (TERM TERM)**

Important **Advice**! In this year's version of the SPL, all numbers in an SPL program are either *float*, or *integer*, however not both "mixed"! Because the **mod** operator works *only* with ***int****eger* numbers, whereas the **div**ision operator works *only* with ***float****ing point* numbers, we introduce the following very "crude" type analysis rule in order *to "make your life easier"* in this practical project:

- **If** a given SPL syntax tree contains both a **mod** node as well as also a **div** node, then the type analyser will *outright* ***reject*** the entire syntax tree, *without the need to go into any finer analysis* of its various sub-branches.
- The corresponding **error message** is: ***"A float-integer-conflict might perhaps be possible"***.

Without this very crude rule you would have to struggle immensely to further *distinguish different* ***sub****-types* of the numeric type from each other throughout a given syntax tree.
Moreover:

- **If** the given SPL syntax tree contains any **mod** node (which forces all numbers to be integers as explained above), then ***no*** NUM token (coming from the lexer) at the leaf-level of the tree may *contain* the *decimal****.dot***

However, if such an outright rejection has not taken place, then the following type checking rules are applicable:

if type_of(TERM) is "numeric" and *if* type_of(TERM) is "numeric",
then **update**: let type_of(TERM) be "**numeric**".

TERM → **add (TERM TERM)**
TERM → **sub (TERM TERM)**
TERM → **mul (TERM TERM)**

if type_of(TERM) is "numeric" and *if* type_of(TERM) is "numeric",
then **update**: let type_of(TERM) be "**numeric**".

TERM → **neg (TERM)** *if* type_of(TERM) is "numeric",
then **update**: let type_of(TERM) be "**numeric**".

BRANCH → **if** BOOL **then { ALGO } else { ALGO }**

if type_of(ALGO) is "ok" and *if* type_of(ALGO) is "ok"
and *if* type_of(BOOL) is "boolean",
then **update**: let type_of(BRANCH) be "ok".

BOOL → **not (BOOL)** *if* type_of(BOOL) is "boolean",
then **update**: let type_of(BOOL) be "**boolean**".

BOOL → **and (BOOL BOOL)**
BOOL → **or (BOOL BOOL)**

if type_of(BOOL) is "boolean" and *if* type_of(BOOL) is "boolean",
then **update**: let type_of(BOOL) be "**boolean**".

BOOL → **eq (TERM TERM)**
BOOL → **larger (TERM TERM)**
BOOL → **lesser (TERM TERM)**

if type_of(TERM) is "numeric" and *if* type_of(TERM) is "numeric",
then **update**: let type_of(BOOL) be "**boolean**".

COND → **while**
COND → **until** **update**: let type_of(COND) be "ok".

---

turn the page ►

LOOP → COND BOOL **do {** ALGO **}**
LOOP → **do {** ALGO **}** COND BOOL

if type_of(ALGO) is "ok" and *if* type_of(COND) is "ok"
and *if* type_of(BOOL) is "boolean",
then **update**: let type_of(LOOP) be "ok".

TO DO:

- Implement the "tree-crawler" algorithm that does the type analysis with help of the symbol table:
- *If* a given SPL syntax tree (coming from the parser) contains a *statically detectable* type error (according to the semantic rules of above), then your type analysis algorithm must emit an appropriate *error message.*
- *If* a given SPL syntax tree (coming from the parser) does *not* contain any *statically detectable* type error (according to the semantic rules of above), then your type analysis algorithm ***may not*** emit any error message: The "ok" syntax tree (together with its corresponding symbol table) will then be **ready for code generation** in the **subsequent phase** of the project ...

And now: **HAPPY PROGRAMMING :)**
